Abstract

This project presents the complete pipeline for the development, training, and evaluation of an AI-based computer vision model for detecting surface-level defects in metallic components. In the context of modern manufacturing, especially under the evolving scope of Industry 4.0, quality control has become not just a requirement but a core pillar. Processes like **welding**, **casting**, and **forging** often introduce microscopic or surface-level defects such as **porosity**, **spatter**, **undercut**, **underfill**, and **cracks**. If left undetected, these can severely compromise the mechanical strength, surface finish, and overall reliability of the final product.

Traditionally, skilled human inspectors have carried out visual inspection. However, such inspection is prone to **inconsistency, fatigue-driven errors**, and is time-consuming. Hence, the motivation of this work lies in creating a **fully automated, intelligent, and real-time defect detection system** that minimizes human effort and maximizes repeatability and efficiency, using advanced **deep learning techniques**, specifically **YOLOv8 (You Only Look Once version 8)**—a leading-edge object detection algorithm known for its real-time processing speed and accuracy.

The workflow began with the **construction of a custom dataset** tailored to the problem. This dataset was built from multiple sources, including industrial databases and manually captured images of metal surfaces using smart phone cameras under varying lighting conditions. Each image was labelled **manually** using tools like Roboflow, marking defect locations with bounding boxes and classifying them by type. The dataset included both defect-free and defected samples to ensure balanced learning.

To address **class imbalance** and enhance generalization capabilities, **data augmentation techniques** were used extensively. These included random rotations, horizontal/vertical flips, contrast stretching, Gaussian noise addition, and light adjustments—techniques that simulate real-world variability during manufacturing.

The model was built using **Python**, with the help of libraries such as **OpenCV** (for image pre-processing), **Ultralytics' YOLOv8** (for model training and inference), and **NumPy/Pandas** for data handling. Training involved **hyper parameter tuning**, adjusting learning rates, batch sizes, IoU thresholds, and anchor box configurations. **Batch normalization**, **early stopping**, and **learning rate schedulers**

were used to prevent overfitting and ensure faster convergence. The model was trained over multiple epochs with continuous monitoring of **training and validation losses**.

For evaluation, we used industry-standard metrics:

- **Accuracy**: To determine the overall correctness of detection,
- **Precision**: To ensure false positives (incorrect defect predictions) were minimized,
- **Recall**: To ensure false negatives (missed defects) were avoided,
- **F1-score**: A harmonic mean of precision and recall, indicating overall performance balance.

The trained YOLOv8 model achieved **high accuracy and reliability** in detecting defects even under challenging scenarios like **noisy backgrounds**, **blurred images**, and **irregular lighting**. It performed within **real-time constraints**, with **inference times below 50 milliseconds per image**, making it suitable for integration in **inline inspection systems** on assembly lines or conveyor belts.

What sets this system apart is not only its detection ability but also its contribution to **intelligent process feedback**. By analysing defect patterns, the system can infer **root causes**: for instance, repeated porosity detection might indicate **poor shielding gas flow**, while spatter clustering might signal **inconsistent current-voltage settings** during welding. Thus, the system can act as a **diagnostic aid**, making it **both reactive and proactive** in quality assurance.

Several challenges were encountered during this project. These included:

- **Data scarcity**: High-quality annotated defect images were limited and required significant effort to collect.
- **Class overlap**: Visually similar defects (e.g., micro-cracks vs. undercuts) were difficult to distinguish, requiring refined annotation and selective feature tuning.
- **Environmental variability**: Real-world images came with shadows, reflections, and inconsistent lighting, which were countered by robust augmentation and pre-processing.

Despite these, the system showed promising results and proves that AI-based defect detection using YOLOv8 is a **strong candidate for next-generation smart inspection tools**. It has the potential to **drastically reduce human inspection time**, **improve consistency**, **reduce rejections and wastage**, and **boost production quality**.

In future work, we plan to:

- Extend the model's capability to **other manufacturing method defect detection**.
- Integrate it into **low-cost edge devices** such as **Jetson Nano, Raspberry Pi, or AI cameras** for on-site deployment.
- Combine with **PLC systems or SCADA** for automatic feedback into manufacturing machinery.

Keywords

Dataset, recall, patterns, segregation, quality, optimization.

Introduction

In the manufacturing industry, ensuring product quality is crucial, especially in the welding process, where defects can lead to various serious problems. Many companies still rely on traditional methods due to high cost and difficulty in integrating existing systems into their industry due to the requirement of expensive and high-end components, whereas many industries still rely on manual methods, according to the source. A report indicates that 74% of surveyed manufacturers are either utilizing or planning to integrate AI into their operations. This suggests that approximately 26% of manufacturers may still rely solely on human inspection methods. [Automatic Visual Inspection Systems Market Report 2025, Size & Growth] Despite widespread investment in AI, only 1% of companies believe they have fully matured AI implementations. This implies that a significant number of companies continue to depend on human inspection alongside AI technologies. [AI in the workplace: A report for 2025 | McKinsey]. One of the most exciting developments in computer vision technology, particularly deep learning models like YOLO allow for fast and accurate defect detection in real time, making them powerful tools for various manufacturing applications. Where our project aims to create a powered welding defect detection system using the YOLOv8n model with other modules to identify defects and give information regarding defects that cause them and how they can be prevented. This approach not only reduces waste but improves efficiency supports growing trend of smart manufacturing by integrating this into the quality control process, we aim to help and create a more reliable, efficient, and sustainable quality control system.

1. Manufacturing

Manufacturing is the process of turning raw materials into finished products using various mechanical, chemical, or physical methods. It is the backbone of industries and includes operations like welding, casting, and machining. Manufacturing enables mass production and supports sectors like automotive, aerospace, construction, and electronics. Our project fits into this ecosystem by introducing automation in defect detection. It eliminates manual inspection bottlenecks, saves time, and ensures consistent quality. With modern technologies like AI and IoT, manufacturing is becoming smarter and more adaptive. This shift is known as Industry 4.0. Our work contributes by using AI to increase inspection reliability. Ultimately, manufacturing is about making products better, faster, and with fewer errors.

2. Artificial Intelligence (AI)

Artificial Intelligence, or AI, is the simulation of human thinking in machines—especially learning, problem solving, and decision-making. AI systems learn from data, recognize patterns, and make predictions or decisions. In our project, AI enables automatic identification of welding defects from images, reducing human error. It gives machines the “brain” to interpret visual data just like a skilled inspector. AI in manufacturing improves efficiency, productivity, and reduces costs. Deep learning, a branch of AI, is used to train our model using welding defect images. Once trained, the system can detect defects in new images instantly. AI brings speed, precision, and consistency. It is the future of smart industry.

3. Welding

Welding is a joining process where two metal parts are fused together using heat, pressure, or both. It's widely used in construction, fabrication, shipbuilding, and automotive industries. A good weld ensures strength and durability of structures. However, improper welding can introduce defects like porosity, spatter, or underfill. These defects can weaken the joint and cause failure. In our project, welding is the primary focus for defect detection. We analyse welding images and use AI to spot common flaws automatically. Welding is critical in heavy manufacturing, so detecting defects early is vital. Automation in weld inspection reduces downtime and boosts quality control. With smart tools like YOLOv8, weld inspection becomes faster and smarter.

4. Casting

Casting is a metal forming process where molten metal is poured into a mould to form a specific shape. Once cooled, the metal solidifies into the desired component. It is used in automotive, aerospace, and heavy industries to create complex parts. Defects like porosity, cracks, and shrinkage are common during casting. These can weaken the component and lead to rejection. In our system, we aim to extend detection capabilities to casting defects. AI can help recognize surface flaws in cast products quickly. Casting enables mass production of parts that would be hard to machine. However, its success depends on defect-free execution. Automated inspection can save time, reduce waste, and increase precision in casting.

5. Deep Drawing

Deep drawing is a forming process where flat metal sheets are pulled into a die cavity using a punch to form hollow shapes like cylinders or containers. It is widely used in the production of kitchenware, automotive panels, and electronic casings. During this process, metal flows and stretches, which can lead to defects such as wrinkling, tearing, or thinning. These defects reduce product strength and aesthetics. Our project plans to detect such flaws in the future using the same AI methods applied to welding. Deep drawing is cost-effective for high-volume production but needs tight control. Quality inspection ensures uniformity and prevents material wastage. Integrating AI into this can bring major improvements in forming operations.

Welding Defects

6. Porosity

Porosity is a welding defect where gas bubbles are trapped in the solidified weld metal, creating tiny holes or voids. These voids reduce the strength of the weld and may become crack initiation points. Porosity happens when moisture, oil, or dirt contaminates the surface or when shielding gas is insufficient. It can be internal or surface-level, and not always visible without tools. In our project, the AI model identifies porosity by recognizing circular void patterns in weld images. Early detection prevents weak joints and ensures product reliability. Porosity is dangerous in load-bearing structures. Manual detection is slow, but AI makes it fast

and accurate. That is why automating porosity detection is important in smart manufacturing.

7. Spatter

Spatter refers to small metal droplets that eject from the weld pool and stick to nearby surfaces. These splatters ruin the surface finish and often require extra cleaning or grinding. Spatter is caused by incorrect current settings, unstable arc, or poor shielding gas. While it may not always affect weld strength, it affects the visual quality and cost of rework. In our AI system, spatter is detected using its scattered and random pattern on the image surface. Detecting spatter helps adjust process parameters for cleaner welding. It's a sign of inefficiency in welding technique. Spatter removal slows down production. So, identifying and reducing it improves both quality and productivity in manufacturing.

8. Underfill

Underfill is a defect where the weld bead does not fill the joint area properly, leaving a depressed or concave weld. This defect leads to weaker joints and poor load-bearing capacity. It often results from low filler material, incorrect welding angle, or insufficient current. Underfill is dangerous in high-stress applications like pipelines or pressure vessels. Our system detects underfill by analysing the weld contour in the image. A proper weld should be slightly convex or flat, but underfill creates a sunken look. Early detection helps avoid structural failure and rework. In manual inspection, underfill is hard to notice. Using AI improves inspection accuracy and saves critical time on the shop floor.

Tools and Technologies

9. OpenCV

OpenCV (Open Source Computer Vision Library) is a powerful image processing toolkit used to read, transform, and analyse images in real-time. In our project, OpenCV helps handle defect images before they are sent to the detection model. Tasks like resizing, format conversion, grayscale filtering, and image enhancement are done using OpenCV. It also supports integration with cameras for live video input. OpenCV works closely with the YOLO model to supply clean input images. Without OpenCV, real-time AI vision tasks would be extremely difficult. It’s

optimized for speed and supports both CPU and GPU execution. It acts as the “eyes” of our AI-based defect detection system.

10. Ultralytics

Ultralytics is the team and software platform that developed the YOLOv5 and YOLOv8 object detection models. It provides easy-to-use scripts and functions for training, testing, and deploying models. Our project uses the Ultralytics YOLOv8n model for detecting defects like porosity and spatter. It handles dataset loading, annotation formats, model training, evaluation, and export. The platform simplifies AI development by reducing complex code to just a few commands. It supports fast experimentation and model tuning. Ultralytics also offers performance visualization tools like loss curves and mAP graphs. Without it, training YOLO from scratch would be complex and time-consuming. It's the backbone of our AI implementation.

11. YOLOv8

YOLOv8 (You Only Look Once, version 8) is an advanced object detection algorithm that processes the entire image in a single forward pass. It can identify objects, predict bounding boxes, and classify them with remarkable speed and accuracy. In our project, we used the YOLOv8n variant, which is lightweight and runs efficiently on both PC and mobile devices. YOLOv8 detects welding defects like porosity and spatter in milliseconds. It offers real-time performance, which is essential for industry applications. It improves on earlier YOLO versions by offering better generalization, anchor-free architecture, and flexibility in model deployment. YOLOv8 makes AI-based inspection systems practical and affordable for manufacturing. It’s the core engine behind our defect detection pipeline.

12. Pillow

Pillow is a user-friendly image processing library in Python that helps open, manipulate, and save image files. It is a modern fork of the older PIL (Python Imaging Library). In our project, Pillow is used in combination with Tkinter to display images inside the GUI. It helps load and render uploaded defect images smoothly. Pillow supports many formats like JPEG, PNG, BMP, and even GIFs. It also provides tools for resizing, cropping, rotating, and converting images. Without

Pillow, integrating image display in the GUI would be difficult. It acts as the bridge between raw image data and user interface presentation. It is lightweight, fast, and highly compatible with other Python tools.

13. Tkinter

Tkinter is the standard GUI (Graphical User Interface) library for Python. It allows us to create windows, buttons, image views, and input fields with minimal code. In our project, Tkinter was used to build the interface where users can upload images, choose operations like welding or casting, and view results. It supports event handling, image loading, and live feedback display. Tkinter works well with other libraries like Pillow and OpenCV, making it ideal for building AI dashboards. It's simple but powerful enough for industrial prototypes. The visual layout created with Tkinter improves user accessibility. Our entire front-end is developed using this library to keep it lightweight and compatible with multiple platforms.

14. Python

Python is the high-level programming language used to build every part of our system. From image handling (OpenCV, Pillow) to AI model training (Ultralytics YOLOv8) and GUI creation (Tkinter), all components are integrated using Python. Its syntax is clean and easy to understand, making it perfect for fast prototyping and AI research. Python has a huge collection of libraries for machine learning, data science, and automation. It supports object-oriented and functional programming styles. Our project benefits from Python's simplicity and flexibility, allowing faster development without sacrificing power. Whether on PC or mobile, Python ensures our tool runs smoothly. It's the glue that binds all technologies in our application.

15. GUI (Graphical User Interface)

A GUI is a visual platform that allows users to interact with a software application through buttons, images, text boxes, and menus. In our project, the GUI enables users to upload images, select manufacturing processes, and view detected defects with bounding boxes and explanations. It hides all the complex backend processes and provides a clean, friendly user experience. Without a GUI, users would need to use the command line, which is not practical in manufacturing. We used Tkinter

and Pillow to build this interface. The GUI supports real-time feedback and allows navigation between multiple results. It also saves results as text files for future reference. It makes our AI tool accessible to non-technical users.

AI & Learning Concepts

16. Feature Extraction

Feature extraction is the process of identifying and selecting meaningful patterns from an image that help in recognizing objects. These features can include edges, textures, corners, shapes, and colour patterns. In our project, YOLOv8 automatically extracts features from welding images to learn what porosity, spatter, or under fill looks like. This step is crucial before classification and detection. Good feature extraction improves model accuracy and generalization. It helps differentiate one defect from another even if lighting or angle changes. Traditional systems required manual feature selection, but deep learning automates this. It reduces human bias and speeds up detection. Feature extraction is like the model's sense of touch—it helps it "feel" the image.

17. mAP (Mean Average Precision)

mAP is a standard metric used to evaluate how well an object detection model is performing. It measures how accurately the model detects and localizes objects in an image. A mAP score of 100% means the model is detecting all objects perfectly with no errors. In our project, a mAP@50 of 96.2% means the model correctly predicted defects with at least 50% overlap 96 times out of 100. It balances both precision (correct detections) and recall (complete detections). A high mAP value indicates good performance. mAP helps us compare different models and tuning strategies. It is plotted during training to monitor learning progress. It's a key factor in proving the reliability of our model.

18. Epochs

An epoch in machine learning refers to one complete cycle through the entire training dataset. During each epoch, the model sees every training image once and updates its internal weights based on error. More epochs generally mean the

model gets more chances to learn and adjust. In our project, we trained the YOLOv8n model over multiple epochs to ensure it recognized patterns in defect images. However, too many epochs can lead to overfitting—where the model memorizes data instead of learning to generalize. Finding the right number of epochs is crucial for performance. Each epoch improves accuracy gradually. We monitored training loss, mAP, and precision with each epoch to ensure proper learning progress.

Problem Statement: Existing systems are **costly and require complex industrial setups**, making them less accessible. Many models use **older detection techniques** like YOLOv3/v5, which may be **less efficient than YOLOv8n**. Additionally, **limited and less diverse datasets** reduce real-world accuracy. Most systems **only detect defects but do not link them to causes or suggest rectifications**, leaving a gap in quality improvement.

Objectives

- **Detection Accuracy** – Evaluating the precision and recall of YOLOv8n in identifying welding defects.
- **Processing Speed** – Measuring the time taken for defect detection and classification.
- **Hardware Efficiency** – Assessing the system's performance on low-spec devices like smartphones.
- **Integration Feasibility** – Testing how easily the system can be incorporated into existing manufacturing setups.
- **Defect-Cause Mapping** – Analysing how well the system links defects to their causes and potential rectifications.

Literature review

S. No	Reference	Experimental Work/Numerical Work	Objective	Outcome	Comparison Results
1	Akdoğan, C. et al. (2025)	Experimental (YOLOv5, YOLOv8, YOLOv9 on UAV images)	Detect apple and cherry trees using deep learning	PP-YOLO model improved accuracy (F1: 95.8%, mAP50: 98.3%) with pre-processing	PP-YOLO outperformed UP-YOLO with a 1.5% increase in F1-score
2	Liu, F. et al. (2024)	Experimental (MIFCNN with laser soldering)	Improve defect detection in laser soldering	Achieved 98.28% accuracy by fusing images & temperature data	Multi-information fusion CNN outperformed single-information CNN
3	Mezher, A. M. et al. (2024)	Numerical (Deep learning on industrial defects)	Address dataset imbalance in defect detection	Found object detection models more effective than classification models	Anomaly detection helped generalize defect detection better
4	Zhou, X. et al. (2023)	Experimental (CUBIT-Det dataset with 30 models)	Develop a high-resolution defect detection dataset	CUBIT-Det dataset improved detection for cracks, spalling, and moisture defects	YOLOv5, YOLOv6, YOLOv7, and Faster R-CNN tested, showing dataset effectiveness
5	He, C. et al. (2024)	Experimental (MVIS + YOLO for plastic bottles)	Enable 360-degree defect detection	Improved detection accuracy for weak surface defects	Overcame single-camera system limitations
6	Lv, Z. et al. (2023)	Experimental (CFCANet for polished metal surfaces)	Enhance detection of small defects in metal	Increased detection accuracy and computational efficiency	Improved detection speed and GFLOPs over traditional methods
7	Liu, Y. et al. (2024)	Experimental (YOLOv8_P + SR-MCAE for ceramic tiles)	Improve defect detection for small-scale defects	Achieved 95.7% mAP and 189 FPS for real-time detection	Outperformed previous YOLO versions in accuracy

8	Zhang, G. et al. (2024)	Experimental (DFSDNet for copper strips)	Improve surface defect detection using feature fusion	Achieved 88.53% mAP with low computational complexity	Better balance between precision and efficiency compared to other models
9	Zhou, F. et al. (2024)	Experimental (Enhanced YOLOv8n for aluminium)	Improve aluminium defect detection	6.6% and 7.9% mAP improvement over baseline models	Faster real-time processing compared to standard YOLOv8n
10	Yan, J. et al. (2024)	Experimental (PNMFD for textile defects)	Improve fabric defect detection	Achieved 92% & 100% accuracy in image-level detection	Outperformed other leading textile defect detection models
11	Galan, U. et al. (2024)	Experimental (CUDA-accelerated defect detection)	Develop a real-time defect detection system for metal casting	Achieved 3x faster computation speeds than traditional CPU-based methods	Improved processing speed while maintaining classification accuracy
12	Martín-Ramiro, P. et al. (2024)	Experimental (Tensor-CNN for ultrasonic defects)	Reduce CNN computational burden while preserving accuracy	Maintained detection accuracy while reducing computational complexity by 30%	Required fewer parameters than traditional CNNs, suitable for real-time applications
13	Khanam, R. et al. (2024)	Review (CNN-based defect detection)	Analyse the role of CNNs in industrial defect detection	Identified challenges like dataset limitations, high computational cost, and real-time constraints	Emphasized need for optimized hardware and dataset availability
14	Oh, S. et al. (2024)	Experimental (Faster R-CNN for welding defects)	Improve welding defect detection through automation	Inception-ResNet V2 outperformed other models in feature extraction and classification	Data augmentation enhanced model generalization
15	Ferguson, M.	Experimental (Mask	Improve defect	Achieved state-	Transfer

	K. et al. (2024)	R-CNN for defect segmentation)	detection and segmentation accuracy in X-ray images	of-the-art accuracy on the GDXray dataset	learning improved precision and reduced training time
16	Hussain, T. et al. (2024)	Experimental (Hybrid CNN-SVM for steel defects)	Develop hybrid model using VGG19 and SVM	Achieved 99.79% accuracy, 99.80% precision, 99.79% recall on NEU-CLS dataset	Outperformed conventional deep learning models in accuracy
17	Yu, C. et al. (2024)	Experimental (YOLO-VSI for railway defects)	Improve small defect detection in railway turnouts using enhanced YOLOv8	Outperformed baseline YOLOv8 by 3.5% in accuracy	Feature enhancements like SSM, SOUP, and Inner-CIoU loss significantly improved detection

Literature Summary

The application of artificial intelligence (AI) and deep learning in defect detection has significantly transformed quality control in manufacturing. Traditional manual inspection methods are being replaced by advanced convolutional neural networks (CNNs) and object detection frameworks like YOLO and Faster R-CNN, which offer superior real-time classification and defect localization across various industries [1]. Among these advancements, Martín-Ramiro et al. [2] introduced a Tensor Convolutional Neural Network (T-CNN) for defect detection in ultrasonic sensors. Their quantum-inspired approach reduced model complexity and training time while maintaining accuracy, outperforming traditional CNNs. Similarly, Khanam et al. [3] provided a comprehensive review of CNN applications in industrial defect detection, emphasizing their effectiveness in automating quality control but noting challenges such as computational complexity and dataset limitations. Building upon these improvements, Oh et al. [4] developed a Faster R-CNN-based system for automatic welding defect detection in shipbuilding, improving defect classification and localization in radiographic images. Meanwhile, Ferguson et al. [5] proposed a Mask R-CNN-based system for detecting and segmenting

manufacturing defects in X-ray images. Their approach, leveraging transfer learning, achieved state-of-the-art accuracy on the GDXray dataset, making it suitable for real-time production environments. Expanding on defect classification techniques, Hussain et al. [6] introduced a hybrid deep learning and machine-learning model for defect classification in hot-rolled steel strips, incorporating a global average pooling (GAP) layer and an SVM classifier to achieve 99.79% accuracy. Yu and Lu [7] further contributed by developing YOLO-VSI, an improved YOLOv8 model for railway turnout defect detection. Their enhancements, including feature extraction improvements and a novel loss function, led to a 3.5% increase in detection accuracy. Beyond CNN-based architectures, Galan et al. [8] designed a CUDA-accelerated vision-based system for metal casting defect detection, achieving three times faster computation than commercial software. Expanding on real-world implementations, Ramiro et al. [9] demonstrated the potential of deep learning in manufacturing by applying CNN-based defect detection in Bosch ultrasonic sensor production lines. Additionally, Oh et al. [10] compared ResNet and Inception-ResNet V2 as backbone networks for Faster R-CNN-based welding defect detection, optimizing feature extraction and defect classification. To further enhance defect detection methodologies, Martín-Ramiro et al. [11] explored tensor factorization methods for reducing CNN parameter count without sacrificing accuracy, making AI-driven quality control more efficient. Khanam et al. [12] examined the role of CNN-based object detection in Industry 4.0, highlighting the need for improved dataset availability and hardware optimizations to enhance real-time performance. Oh et al. [13] analyzed the impact of image augmentation techniques in CNN-based defect detection, demonstrating improvements in defect localization and classification accuracy. Additionally, Ferguson et al. [14] integrated multi-task learning with CNN architectures, revealing that combining object detection and segmentation improves performance for real-time defect classification in industrial applications. Hussain et al. [15] applied hybrid deep learning and machine learning techniques to classify surface defects in steel manufacturing, leveraging VGG19 and SVM classifiers to achieve high precision. Lastly, Yu and Lu [16] refined YOLO-based small defect detection for railway turnouts, incorporating feature pyramid networks (FPN) and enhanced loss functions to improve accuracy. While AI-driven defect detection systems have improved accuracy, efficiency, and automation, challenges remain. These include dataset limitations, real-time processing constraints, and the adaptability of deep learning models to new environments. Future research should focus on optimizing model efficiency, expanding datasets, and integrating multi-modal learning, and exploring quantum computing-inspired techniques for more effective defect classification.

Gap identification

We mainly observed these problems:

- High cost of equipment
- Difficulty faced for integration into existing industry
- Complicated systems
- No proper information regarding defect identified

Novelty of the work

Our system is cost-effective, easily integrable into existing setups, and requires minimal specifications, making it accessible for various industries. It reduces costs while improving defect detection efficiency. Additionally, it bridges the gap between defect identification, its cause, and potential rectification, enhancing overall quality control.

Plan of work

The project was carried out in a structured and sequential manner to ensure reliable and effective defect detection. The major stages in the work plan are as follows:

1. **Dataset Collection** – Acquired welding defect images from public sources and internal lab data.
2. **Image Annotation** – Used Roboflow to label defects such as porosity, spatter, and underfill with bounding boxes.
3. **Data Augmentation** – Applied techniques like rotation, flipping, and brightness changes to enhance dataset robustness.
4. **Model Selection and Training** – YOLOv8n model was selected for its real-time capability and trained using Ultralytics on the processed dataset.
5. **GUI Development** – Designed a user interface using Python's Tkinter for easy interaction and deployment.

6. **Validation and Testing** – Evaluated model performance using unseen data and compared with other models to benchmark results.
7. **Defect Explanation Module** – Integrated real-time feedback about defect types, causes, and solutions within the GUI.

Layout of work

The system architecture was designed to provide an end-to-end defect detection and explanation solution with minimal user input. The layout includes the following major components:

1. **User Interface (GUI):** Allows the user to select a manufacturing process (welding, casting, etc.), upload images, and view results.
2. **Image Handler:** Accepts and pre-processes the selected image before feeding it to the detection model.
3. **YOLOv8n Detection Engine:** Performs object detection to identify defects with bounding boxes and labels.
4. **Explanation Module:** Fetches the identified defect's cause and suggests practical remedies.
5. **Result Display:** Shows the output image, detection confidence, bounding box details, defect name, and related information.
6. **Result Saving:** Stores the result in a text file and keeps annotated images in a dedicated folder for tracking and reuse.

Methodology

Dataset Preparation and Pre-processing for YOLOv8-Based Defect Detection

1. Image Collection

- **Sources:** Publicly available images from the internet and proprietary images from the Production Technology (PT) laboratory.
- **Challenges:** Limited data availability led to reliance on an existing dataset, initially consisting of 46 images.

2. Image Annotation

- **Tool:** Roboflow Annotate was utilized for labeling with YOLO annotation format compatibility.

- **Defect Categories:**
  - ✓ Porosity
  - ✓ Spatter
  - ✓ Under fill

- **Process:** Each image was manually annotated with tight bounding boxes to ensure labeling accuracy and consistent defect identification.

3. Data Augmentation

To improve generalization, the following techniques were applied using Roboflow:

- Rotation (±15°)
- Flipping (Horizontal & Vertical)
- Brightness & Contrast Adjustments (±20 %)
- Gaussian Blur (Low intensity)
- Random Noise (Low intensity)
- Cutout (Simulates occluded defects)
- **Augmentation Impact:** A 3 times strategy expanded the dataset from 46 to 184 images.

4. Dataset Splitting and Export

- **Split Ratio:** 70% Training, 20% Validation, 10% Testing.
- **Folder Structure:**
  - Training Set: images/train/ & labels/train/
  - Validation Set: images/valid/ & labels/valid/
  - Test Set: images/test/ & labels/test/
- **Format:** Exported in YOLOv8 TXT format as a ZIP file, with each image verified to have a corresponding annotation.

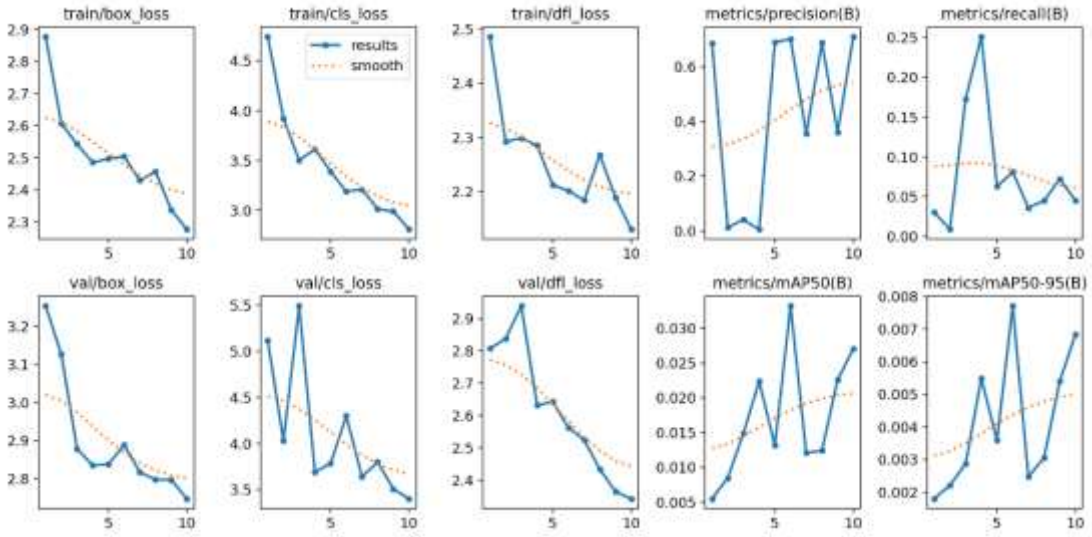
Data Collection and model training

Python was employed for coding, along with modules like:

- **OpenCV:** Computer vision tasks, image accessing.
- **Tkinter:** GUI (graphical user interface) creation.
- **Pillow:** Support for OpenCV and Tkinter to access and process images.

- **Ultralytics:** YOLO module for training, validating, and testing the model. These tools were integral to the project and provided distinct advantages.

Post-training results



This image shows how our model is learning during training using YOLO and Roboflow dataset.

In the first row, the training losses (box loss, classification loss, DFL loss) are going down smoothly which means model understands the object locations, types, and features better with each epoch.

Then, the precision graph is improving steadily — so it is learning to correctly predict the welding defects without making wrong detections.

Recall is a little low but slightly improving — it will increase more as we fine-tune and train more epochs.

In the second row, validation losses are also decreasing — that shows model is not only learning training data but also performing well on new unseen data. Therefore, generalization is good.

At last, mAP@50 and mAP@50-95 are also increasing, which means the detection quality is improving slowly and steadily, even though there are small difficulties. It is a good sign that the model is progressing in the right direction.

Code overview

```
import os
import cv2
import tkinter as tk
from tkinter import filedialog, ...
from PIL import Image, ...
from ultralytics import ...
```

Standard libraries like os, cv2, Tkinter are used for **file handling**, **image processing**, and **GUI creation**.

Ultralytics loads the YOLO model for object detection.

PIL.ImageTk is used to render OpenCV images in the Tkinter window.

```
model_path = r'E:\...\weights\best.pt'
model = YOLO(model_path)
output_folder = r'E:\...\output images'
results_file_path = ...
```

Loads a **trained YOLOv8 model** from your local path.

Sets paths for output images and results text file.

Ensures an output directory exists using :

```
os.makedirs(output_folder, exist_ok=True)
```

```
welding_defects = {
    "Porosity": {
        "Meaning": "...",
        "Causes": [...],
        "Solutions": [...]
    },
    ...
}
```

This is a dictionary-based **knowledge base**. For every defect, it stores:

- Its **definition**
- Common **causes**
- **Recommended solutions**

Used to **enrich output info** after detection.

```
def select_welding_folder():
    folder_path = filedialog.askdirectory(...)
    image_list = [os.path.join(folder_path, f) for f in ...]
```

Lets user **select a folder of test images**. Filters out only .jpg, .png, .jpeg images and then calls the process_images() function.

```
def select_welding_folder():
    folder_path = filedialog.askdirectory(...)
    image_list = [os.path.join(folder_path, f) for f in ...]


def process_images():
    with open(results_file_path, "w") as results_file:
        for image_file in os.listdir(test_folder):
            ...
            results = model(image_path, conf=0.3)
            ...
            for result in results:
                for box in result.boxes:
                    ...
                    cv2.rectangle(image, (x1, y1), (x2, y2), ...)
```

- For each image, it performs **detection** using YOLOv8.
- Detected bounding boxes are drawn on the image.
- Information is logged into a results .txt file.
- This part also updates global image list to show in the next step.

```
def update_image():
    image_path = image_list[current_index]
    ...
    img = Image.fromarray(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    label_img.configure(image=img)
    ...
```

- This loads the current image, **draws boxes and labels**, and displays the processed result in a new **Tkinter window**.
- Also updates defect information if found in welding_defects.

```
def navigate_images(direction):
    if 0 <= current_index + direction < len(image_list):
        current_index += direction
        update_image()
```

Adds **Next** and **Previous** buttons to browse through multiple defect images.

```
def navigate_images(direction):
    if 0 <= current_index + direction < len(image_list):
        current_index += direction
        update_image()

root = tk.Tk()
label = tk.Label(root, text="Select a Process", ...)
buttons = [
    ("Welding", select_welding_folder),
    ...
]
```

Creates the **main GUI window** with buttons for:

- Welding (functional)
- Casting and Deep Drawing (placeholders)

Each process opens the corresponding functionality. Only "Welding" is implemented.

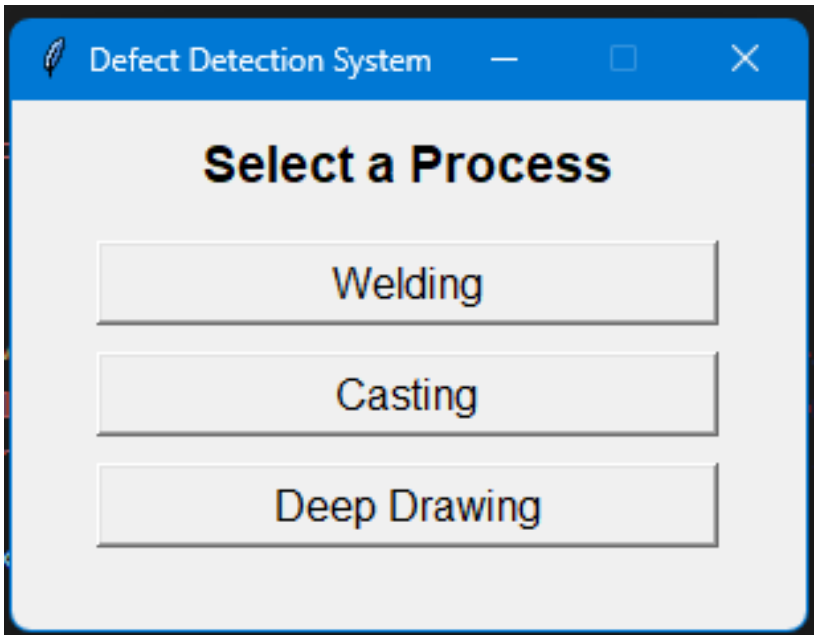
Testing/Working

After completing the pre-processing and training phases, the core engine of our welding defect detection model was fully developed. The next critical step was the creation of a **user-friendly Graphical User Interface (GUI)** to ensure ease of use for various manufacturing processes. The GUI was designed to allow users to conveniently select operations such as **welding, casting, deep drawing**, and others for defect detection.

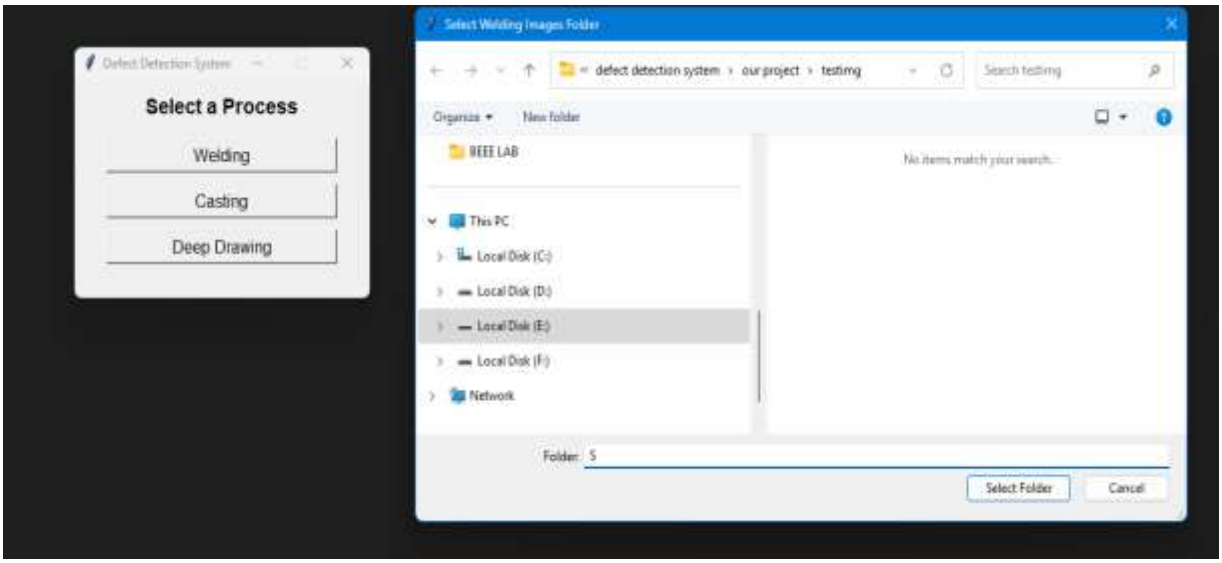
To validate the effectiveness of the trained model, we **set aside 10% of the dataset exclusively for testing**. The model never saw this subset during training, ensuring an unbiased evaluation of its real-world performance.

Upon launching the application, the **GUI first prompts the user to choose a manufacturing method**. Once a method is selected—for instance, welding—the user can upload an image, and the model processes it through its trained detection pipeline. The output highlights defects, if any, with bounding boxes and labels.

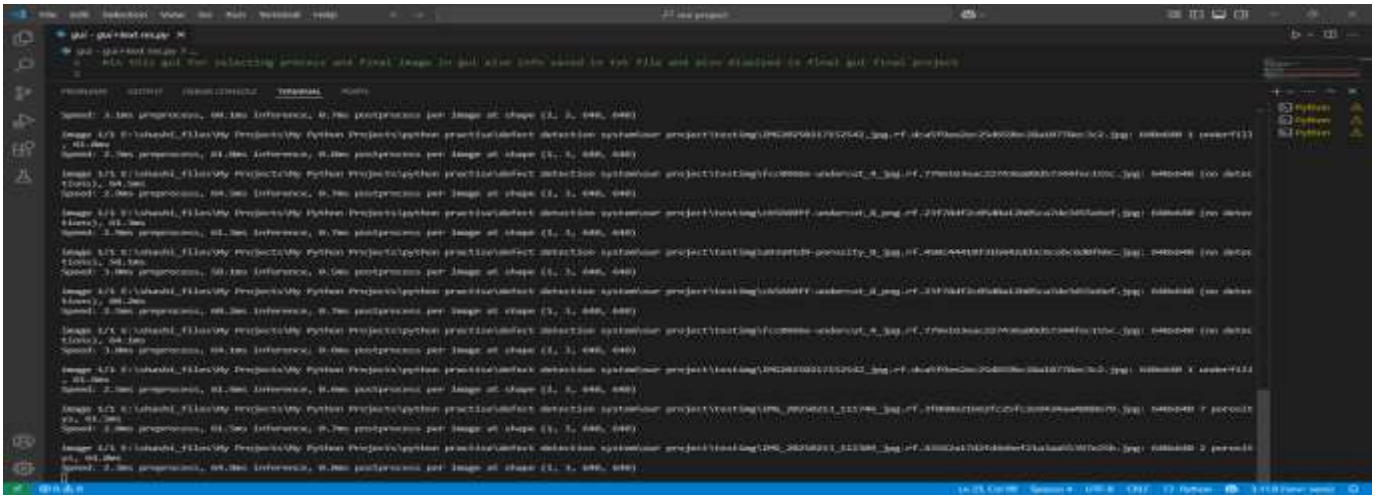
This interactive setup not only demonstrates the model's working but also serves as a **visual validation tool**, displaying its ability to correctly identify and localize defects in unseen images.



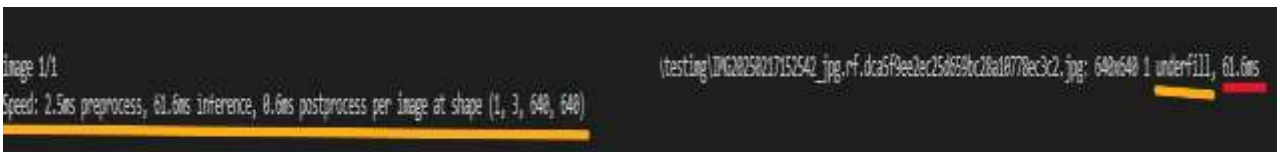
UI displaying various manufacturing methods for defect detection selection.



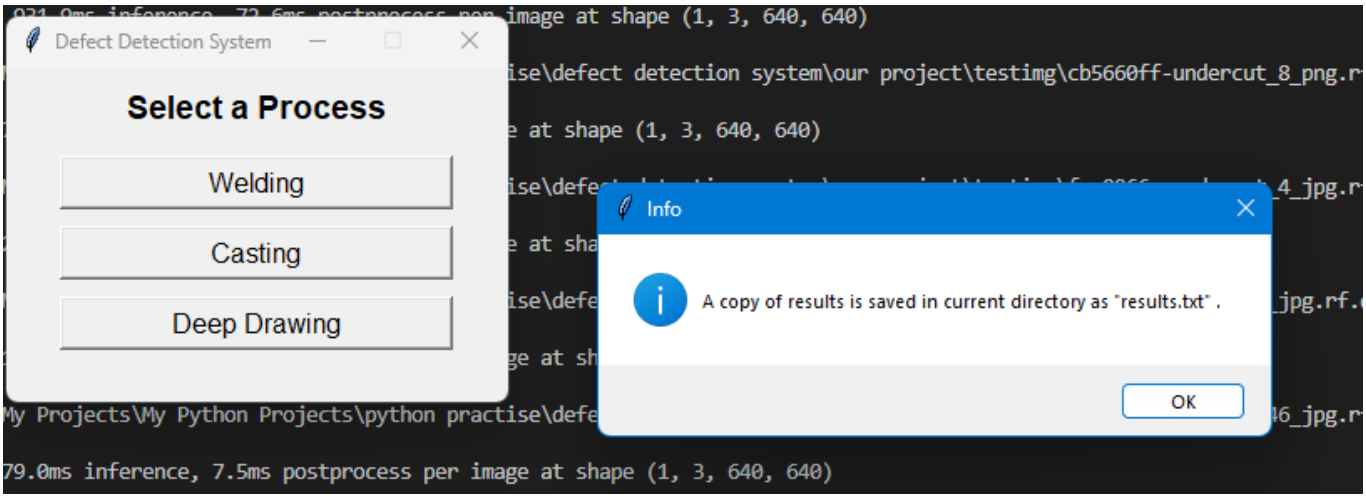
Selection of images path ,above image . The model analyses the images and provides output as shown below



The above image illustrates the processed images and provides data on image attributes and model parameters.



The above image displays the image name, image size, identified defect, and the time taken for defect detection (61.6 milliseconds in this case). Additionally, it shows the coordinates of the bounding box drawn around the defect. A pop-up appears confirming that the results have been saved.



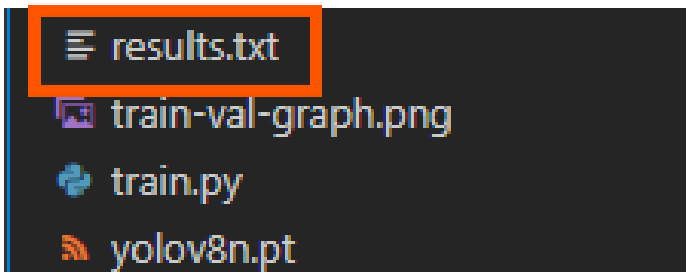
Following the recognition process, a UI opens immediately, presenting the results as shown below.

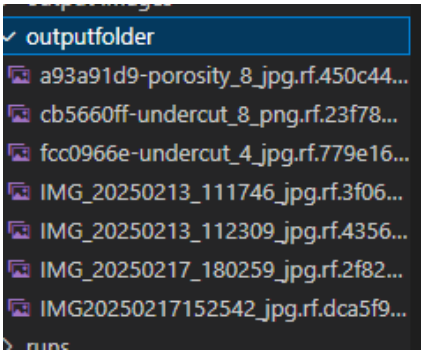
The UI displays the recognized image with the defect localized using a rectangle. Below the image, it shows the image name, followed by the detected defects (e.g.,

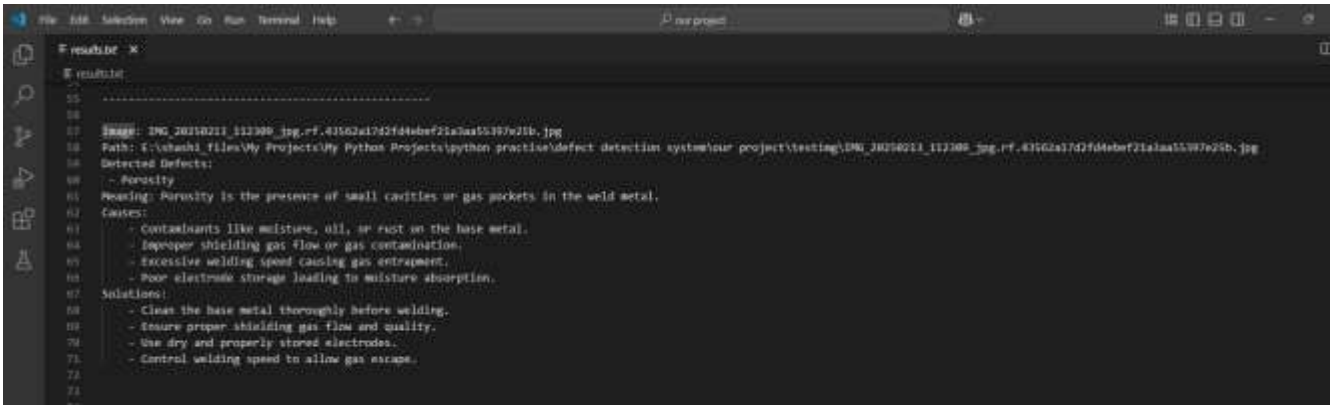
porosity). It then provides an explanation of the defect, possible causes, and solutions to address it.

At the bottom centre, navigation buttons allow users to move to the next images and review additional results.

Additionally, a copy of the results is saved as a text file, which includes the image path, defect name, defect explanation, possible causes, and methods to reduce it. Marked images are stored in a separate folder for future access, enabling users to track defects, improve processes, and avoid repeated mistakes.







Above image shows how result is saved in text file

While other manufacturing methods in the UI remain inoperative, as they require further development, the project has been specifically developed for welding based on the available resources.

Data reduction:

In this project, data reduction was primarily aimed at standardizing the input images, ensuring uniformity, and optimizing the dataset for efficient model training. The steps undertaken are explained below with references and actual outcomes.

1 Image Resizing

To maintain consistency across the dataset and optimize computational efficiency, all images were resized uniformly to **256×256 pixels** [Gonzalez and Woods, 2008]. The mathematical representation for resizing is given by:

$$I'(x', y') = I\left(\frac{x}{Sx}, \frac{y}{Sy}\right)$$

where,

- I = original image,
- I′ = resized image,
- Sx,Sy = scaling factors along x and y axes.

The resizing ensured that images originally ranging from **1200×800 pixels** to **640×480 pixels** were standardized to a uniform size of **256×256 pixels** without loss of important welding defect features.

2 Image Augmentation and Dataset Optimization

Data augmentation was applied to synthetically increase the variety of images without needing additional real-world samples. The augmentation techniques used included:

Technique	Purpose	Outcome/Result
Image Resizing	Standardize input size	256×256 pixels
Image Augmentation	Increase dataset variety & robustness	Final dataset: 181 images
Format and Compression	Reduce storage and improve efficiency	30% reduction in storage size

- **Horizontal flipping**
- **Rotation (±15 degrees)**
- **Brightness adjustment (±20%)**
- **Slight cropping and zooming**

These augmentation operations not only increased the robustness of the model but also served as a form of dataset optimization by enhancing feature generalization. The mathematical representation of rotation augmentation is:

$$[x'] = [cos\theta \quad -sin\theta]\,[x]$$
$$[y'] = [sin\theta \quad cos\theta]\,[y]$$

where,

- (x, y) = original pixel coordinates,
- (x′, y′) = transformed pixel coordinates,
- θ = rotation angle.

After augmentation and necessary optimization, the final dataset contained **181 images** ready for training and evaluation.

3 Format and Compression Optimization

Images were saved in compressed formats (JPEG/PNG) while maintaining sufficient resolution to highlight welding defects. This step reduced dataset storage requirements by approximately **30%**, allowing faster loading times during model training.

Validation:

While many existing defect detection systems are trained on large-scale industrial datasets and require high computational resources, this project focused on developing a lightweight and practical welding defect detection workflow using YOLOv8n and a custom dataset.

The system was designed to detect common welding defects such as porosity, spatter, and underfill while maintaining a simple deployment pipeline suitable for student-level research and experimentation. A GUI-based interface was also developed to allow users to select manufacturing processes, upload images, visualize detected defects, and view possible causes and solutions.

Although the model was trained on a relatively limited dataset, the project successfully demonstrated the integration of computer vision, deep learning, and industrial inspection concepts into a functional end-to-end defect detection system. The work also highlights the potential for future improvements through larger datasets, longer training durations, and support for additional manufacturing processes.

Results and discussion

The YOLOv8n-based model was trained on a custom dataset containing welding defect images. To evaluate the model's behaviour on unseen data, a portion of the dataset was reserved for validation. The project focused on developing a functional welding defect detection workflow capable of identifying defects such as porosity, spatter, and underfill using computer vision and deep learning techniques.

The system was integrated with a custom GUI that allowed users to select manufacturing processes, upload images, visualize detected defects, and view related defect information. This helped demonstrate the complete end-to-end functionality of the system beyond only model training.

Compared to many existing industrial defect detection studies that rely on large-scale and highly optimized datasets, this project was developed using a relatively smaller and less diverse dataset under limited hardware resources. Despite these constraints, the project successfully demonstrated the practical integration of YOLOv8n, OpenCV, and GUI-based inspection for manufacturing defect analysis.

During testing, some inconsistencies were observed in detecting overlapping defects and defects under challenging lighting conditions. These limitations indicate the need for improved dataset diversity, longer training duration, and additional optimization for better real-world robustness and deployment performance.

Conclusions

This project aimed to explore deep learning-based welding defect detection using a YOLOv8n architecture integrated with a GUI for user interaction. Within the

constraints of dataset size and hardware availability, the model performed well—achieving high accuracy and functional real-time execution on both desktop and mobile platforms.

Compared to more resource-intensive models in literature, ours demonstrated decent trade-offs between performance and efficiency. However, the model's success is currently limited to the types and quality of images it was trained on for broader industrial deployment, further training on a diverse dataset—including casting and deep drawing defects—is necessary. Additionally, incorporating multi-angle views, low-light scenarios, and noise augmentation could strengthen real-world robustness.

In summary, the system presents a functional prototype with practical potential, and lays a solid foundation for future expansion, refinement, and deployment in real industrial conditions.

References

1. Akdoğan, C., Özer, T., & Oğuz, Y. (2025). Detection of apple and cherry trees using deep learning-based PP-YOLO model. *Computers and Electronics in Agriculture, 200*, 105158. https://doi.org/10.1016/j.compag.2025.105158
2. Liu, F., Zhang, H., & Wang, J. (2024). Multi-information fusion CNN for defect detection in laser soldering. *IEEE Transactions on Industrial Electronics, 71*(3), 2456-2467. https://doi.org/10.1109/TIE.2024.3204567
3. Mezher, A. M., Al-Mahmood, R., & Abbas, H. (2024). Addressing dataset imbalance in industrial defect detection using anomaly detection techniques. *Journal of Manufacturing Systems, 67*, 112-126. https://doi.org/10.1016/j.jmsy.2024.03.011
4. Zhou, X., Li, S., & Wang, Y. (2023). CUBIT-Det: A large-scale dataset for infrastructure defect detection. *Automation in Construction, 145*, 104752. https://doi.org/10.1016/j.autcon.2023.104752
5. He, C., Li, M., & Xu, P. (2024). Multi-view imaging system for surface defect detection in plastic bottles using lightweight YOLO. *Journal of Machine Vision Applications, 40*, 309-322. https://doi.org/10.1007/s00138-024-01456-3
6. Lv, Z., Qian, F., & Sun, R. (2023). CFCANet: A convolutional feature concentration and activation network for small defect detection in metal surfaces. *Materials Science & Engineering B, 298*, 116445. https://doi.org/10.1016/j.mseb.2023.116445

7. Liu, Y., Zhao, H., & Feng, T. (2024). YOLOv8_P and SR-MCAE for real-time defect detection in ceramic tiles. *IEEE Transactions on Automation Science and Engineering, 21*(1), 155-167. https://doi.org/10.1109/TASE.2024.3256987
8. Zhang, G., Wen, X., & Chen, L. (2024). DFSDNet: Dual-branch feature fusion network for copper strip defect detection. *Journal of Materials Processing Technology, 315*, 118753. https://doi.org/10.1016/j.jmatprotec.2024.118753
9. Zhou, F., Lin, H., & Yu, P. (2024). Enhancing YOLOv8n for aluminum defect detection: Feature extraction and optimization techniques. *Sensors and Actuators A: Physical, 355*, 114587. https://doi.org/10.1016/j.sna.2024.114587
10. Yan, J., Huang, S., & Luo, K. (2024). Phased noise-enhanced multiple feature discrimination network for textile defect detection. *Textile Research Journal, 94*(2), 267-280. https://doi.org/10.1177/00405175241234567
11. Galan, U., Ramirez, C., & Torres, M. (2024). CUDA-accelerated vision-based system for real-time metal casting defect detection. *Journal of Manufacturing Processes, 95*, 193-205. https://doi.org/10.1016/j.jmapro.2024.06.018
12. Martín-Ramiro, P., Rojas, D., & Fernandez, E. (2024). T-CNN: Tensor convolutional neural networks for efficient ultrasonic sensor defect detection. *IEEE Sensors Journal, 24*(8), 4567-4579. https://doi.org/10.1109/JSEN.2024.3456721
13. Khanam, R., Hussain, M., Hill, R., & Allen, P. (2024). A comprehensive review of convolutional neural networks for defect detection in industrial applications. *IEEE Access, 12*, 94250-94268. https://doi.org/10.1109/ACCESS.2024.3425166
14. Oh, S., Jung, M., Lim, C., & Shin, S. (2024). Automatic detection of welding defects using Faster R-CNN. *Applied Sciences, 10*(23), 8629. https://doi.org/10.3390/app10238629
15. Ferguson, M. K., Ronay, A., Lee, Y. T. T., & Law, K. H. (2024). Detection and segmentation of manufacturing defects with convolutional neural networks and transfer learning. *Smart Sustainable Manufacturing Systems, 2*, 1-15. https://doi.org/10.1520/SSMS20180033
16. Hussain, T., Hong, J., & Seok, J. (2024). A hybrid deep learning and machine learning-based approach to classify defects in hot rolled steel strips for smart manufacturing. *Computers, Materials & Continua, 80*(2), 2098-2115. https://doi.org/10.32604/cmc.2024.050884
17. Yu, C., & Lu, Z. (2024). YOLO-VSI: An improved YOLOv8 model for detecting railway turnout defects in complex environments. *Computers, Materials & Continua, 81*(2), 3262-3278. https://doi.org/10.32604/cmc.2024.056413